Lab: RAG Conversazionale (memoria & sessioni)

Modulo 3 · Lezione 3



La lezione scorsa: single-turn per costruzione

```
indicizza()
```

```
recupera(query, k)
```

```
rispondi(query)
```

`scaffolding/aikit/rag.py` — da oggi nel toolkit

```
D: Perché il piano si ferma sempre a metà corsa?
→ …memoria di calibrazione corrotta […] [ticket-supporto-003] [ticket-supporto-001]

D: E per il secondo caso?
→ Non lo trovo nei documenti.
```

> `rag.py` funziona, cita le fonti, si rifiuta quando non sa. Ma dentro `recupera()` entra solo la domanda così com'è: al primo follow-up fallisce. Oggi entra nel toolkit, si modifica una volta insieme (la generazione diventa una funzione a sé) e poi si importa: il file di lavoro dei lab è `chat.py`.



Agenda

	Blocco	Dove	Tempo
	Brief: il follow-up e le quattro cose di oggi	slide	25′
	Live coding: rag.py, la generazione diventa una funzione	`scaffolding/aikit/rag.py`	10′
	Lab 1: la memoria si aggancia al RAG	`working/chat.py` · TODO 1–2	45′
	Lab 2: query condensation	`working/chat.py` · TODO 3	60′
	Pausa	—	15′
	Lab 3: summary memory e sessioni	`working/chat.py` · TODO 4–5	50′
	Blocco flessibile	slide / `working/chat.py`	35′

Dieci minuti su `rag.py` insieme, poi un solo file di lavoro: `working/chat.py`. I dialoghi li dà il docente, uno alla volta.



Dal vivo: il follow-up sulla pipeline single-turn

```
D: Perché il piano si ferma sempre a metà corsa?

→ …una memoria di calibrazione corrotta, caso tipico dopo un'interruzione di corrente […] [ticket-supporto-003]
[ticket-supporto-001]

D: E per il secondo caso?

  1. manuale-lumadesk-014  score=0.388  Se il display resta spento, prova un'altra presa…
  2. guida-smart-working-006  score=0.380  Da casa valgono le stesse regole dell'ufficio…
  3. guida-smart-working-010  score=0.359  Lo smart working non tocca inquadramento…
  4. guida-smart-working-009  score=0.348  Il remoto ordinario si intende dall'Italia…

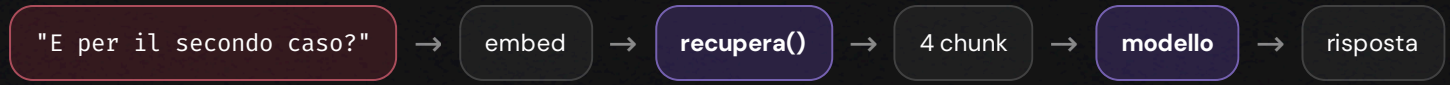
→ Non lo trovo nei documenti.
```

Quattro chunk recuperati, nessun ticket. Il retrieval ha cercato letteralmente "**secondo caso**".

esegue il docente
scaffolding/demo_brief.py



Il retrieval non ha una storia



~~storia della conversazione~~

non entra da nessuna parte: né nel retrieval, né nel prompt

```
# chi è vicino a «secondo caso»
manuale-lumadesk-014   0.388
guida-smart-working-006   0.380
ticket-supporto-003   fuori dai primi 4
```

> Anche se il modello vedesse la storia, l'embedding di «E per il secondo caso?» non assomiglia a nessun ticket. Il problema non è il modello: sono le **parole con cui cerchiamo**.

Le quattro cose di oggi

1	**La storia nel prompt** — il modello vede i turni precedenti, con un budget di token	`rispondi_in_chat()` · `tronca()`	Lab 1
2	**Query condensation** — il follow-up si riscrive usando la storia, ed è quello che va al retrieval	`condensa()`	Lab 2
3	**Summary memory** — la parte vecchia della storia diventa un riassunto invece di sparire	`riassumi()`	Lab 3
4	**Sessioni** — la storia su disco, con un nome, per riprendere dopo aver chiuso il programma	`salva_sessione()` · `carica_sessione()`	Lab 3

Niente framework: la memoria è codice esplicito, in un solo file.



Cosa entra nel modello con una storia

`instructions — le regole del prompt RAG, per tutta la conversazione`

`user — Quanto costa la LumaDesk LD-210?`

`assistant — 849 euro [listino-catalogo-002].`

`user — <documenti> i 4 chunk </documenti> <domanda> … </domanda>`

turno 1 · turno 2 · … · il messaggio del turno, con i chunk dentro

IN STORIA NON VANNO

i chunk. In storia vanno **domanda e risposta** di ogni turno: i documenti entrano nel prompt del turno e basta. Altrimenti in tre turni si rispediscono dodici chunk a ogni chiamata.

LA STORIA PESA

si conta in token e si tiene in un budget.

```
# dentro rag.genera(), tra poco
r = client.responses.create(model=CONFIG["modello"], instructions=ISTRUZIONI,
                            input=storia + [{"role": "user", "content": prompt}])
```



La memoria di Modulo 2 · Lezione 3 diventa funzioni

```
# chatbot.py, Modulo 2 · Lezione 3
class Chatbot:
    self.storia = []
    def send(self, testo): …
    def _tronca(self): …

# conta_token con tiktoken
```

```
# working/chat.py, oggi
def tronca(storia): …
def rispondi_in_chat(storia, domanda): …

    chunks = rag.recupera(domanda)
    risposta = rag.genera(chunks, domanda, storia)

# conta_token(storia) è dato
```

> La memoria l'abbiamo già scritta: sta nella classe `Chatbot` di uno script che non tocchiamo da dieci lezioni, non nel toolkit. Oggi si **estrae** in due funzioni piatte e si aggancia a `rag.recupera()` e `rag.genera()`.



Buffer memory: il troncamento per budget

budget 350 token

~~turno 1~~ ~~turno 2~~ | turno 3 | turno 4 | turno 5

[buffer] tolti i 4 messaggi più vecchi

come

finché la storia supera il budget si toglie il messaggio più vecchio, intero (tiktoken, Modulo 2 · Lezione 2)

CONFIG["budget_token"] = 350

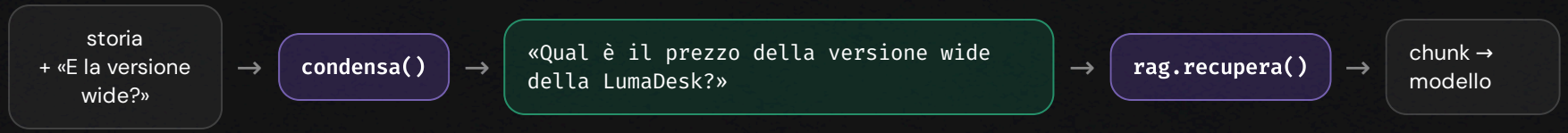
stretto apposta: dopo tre turni la storia va da 56 a 324 token. In produzione sarebbero migliaia

cosa si perde

tutto ciò che stava nei turni caduti, anche quello che il prossimo follow-up richiama



Query condensation: riscrivere prima di cercare



in storia resta l'originale: la storia è quello che è stato detto

```
# il prompt di condensa()
Riscrivi la nuova domanda in modo che sia comprensibile
DA SOLA: sostituisci pronomi e riferimenti con ciò a cui
si riferiscono. Se è già autonoma, ricopiala uguale.
Rispondi SOLO con la domanda riscritta.
```

> È un problema di **prompting**: il retrieval non si tocca, gli si dà una domanda migliore.

`+1 chiamata a turno` — la riscritta va anche nel prompt del turno, così la risposta è coerente coi chunk



La domanda riscritta, misurata

D: Quanto costa la LumaDesk LD-210? → 849 euro [listino-catalogo-002] · poi: «E la versione wide?»

NUDO (fine Lab 1)

1. changelog-rilasci-002 0.399
2. changelog-rilasci-010 0.398
3. changelog-rilasci-005 0.384

→ Non lo trovo nei documenti.

CONDENSATO (Lab 2)

↳ cercato: Qual è il prezzo della versione wide della LumaDesk?

1. listino-catalogo-002 0.648
2. manuale-lumadesk-000 0.585

→ La versione wide, LD-210W, costa 949 euro [listino-catalogo-002].

La riscrittura usa la **risposta precedente del modello**: se era vaga, la riscritta è vaga.

Si legge sempre la riga ↳ cercato: prima di giudicare i chunk.



Summary memory: riassumere invece di tagliare

buffer

~~turno 1~~ ~~turno 2~~ ~~turno 3~~ turno 4 turno 5

riassunto

Riassunto della conversazione finora: … turno 5

come

sopra budget, tutti i messaggi tranne l'ultimo turno diventano **un** messaggio di riassunto scritto dal modello, con fatti e fonti. Un riassunto vecchio viene assorbito nel nuovo

si tiene

i fatti di dieci turni fa, in poche righe

si perde / costa

dettaglio e tono; ogni riassunto è una chiamata. `CONFIG["memoria"]`: "buffer" o "riassunto"



Sessioni: la storia su disco




```
sessione 'prova': 4 messaggi ripresi
```

> Riprendere una conversazione: la storia è su disco, con un nome. Un file JSON per sessione, salvato a ogni turno, ricaricato all'avvio. Chiuso e riaperto con lo stesso nome, il follow-up sul turno di prima funziona ancora.

Esporre le sessioni via API è materia del Modulo 5.



rag.py: la generazione diventa una funzione a sé

OGGI — rispondi(query)

```
chunks = recupera(query)
prompt = regole + <documenti> + <domanda>
input=[{"role": "user", "content": prompt}]
```

tre cose in una: chunk e storia non entrano dall'esterno

DOPO IL LIVE CODING

```
ISTRUZIONI = "le regole del prompt"
def genera(chunks, domanda, storia=None):
    prompt = <documenti> + <domanda>
    input=storia + [messaggio del turno]
def rispondi(query, storia=None):
    chunks = recupera(query)
    return genera(chunks, query, storia), chunks
```

Chi ha chunk diversi (hybrid search, re-ranking) o una storia (`chat.py`, oggi) chiama `genera()` con i suoi. La generazione non si ricopia; `rispondi()` single-turn resta com'era.

Un modulo di `aikit/` si modifica così: in aula, con la diff minima che serve alla lezione, mai in silenzio.



Live coding: rag.py, genera() e rispondi()

REPLICATE — scrive il docente, voi con lui

`scaffolding/aikit/rag.py` · 10 minuti · riferimento: `solutions/aikit/rag.py`

si rientra dopo il rilancio della demo del brief

`1 · ISTRUZIONI`

le regole del prompt escono dal prompt: valgono per ogni chiamata, single-turn o con la storia

`2 · genera(chunks, domanda, storia=None)`

il blocco documenti + domanda, la chiamata con instructions=ISTRUZIONI e la storia davanti nell'input

`3 · rispondi(query, storia=None)`

recupera(query) e poi genera(); ritorna ancora (risposta, chunk usati)

`FATTO QUANDO`

Rilanciando `python scaffolding/demo_brief.py` (esegue il docente) escono le stesse risposte di prima: la baseline non è cambiata. Chi resta indietro copia `solutions/aikit/rag.py` al posto del proprio e va avanti. È l'unica modifica a `rag.py` di oggi: nei lab non si tocca più.



Lab 1: la memoria si aggancia al RAG

SCRIVETE VOI — da soli, con affiancamento

`working/chat.py` · `TODO 1-2` · `45 minuti` · `consegna in CONSEGNA.md`

si rientra sulle slide per il debrief

`TODO 1 · tronca(storia)`

il troncamento di Modulo 2 · Lezione 3 come funzione: toglie i più vecchi finché la storia sta nel budget

`TODO 2 · rispondi_in_chat(storia, domanda)`

rag.recupera() → rag.genera(chunks, domanda, storia) → storia += domanda e risposta → tronca()

`FATTO QUANDO`

Cinque turni restano sotto il budget (la riga `[buffer]` compare almeno una volta) e le risposte hanno ancora le citazioni. Il follow-up fallisce ancora (rifiuto, o fonte sbagliata): è atteso, si cura nel Lab 2. I dialoghi li dà il docente, un dialogo alla volta; per verificare da soli basta la coppia della slide 4. Il file ha già CONFIG, conta_token() e il loop; al nome della sessione, per ora, invio. Se all'avvio esce `AttributeError … 'genera'`, il vostro rag.py è rimasto indietro: copiate `solutions/aikit/rag.py` al suo posto. Setup e primo avvio (indicizza, ~2 min) nel README.



Dopo il Lab 1: il generatore ricorda, il retrieval no

D: E dall'estero? → 2 settimane l'anno, 30 giorni di anticipo [guida-smart-working-009]

D: Con quanto preavviso devo chiederlo?

1. policy-resi-garanzia-003 0.463
2. guida-smart-working-004 0.453

→ Devi chiedere l'autorizzazione con almeno 30 giorni di preavviso ~~[guida-smart-working-008]~~

il fatto viene dalla storia, la fonte no

D: Quale versione lo corregge? → la 2.3.0 [changelog-rilasci-002]

D: Come la installo?

1. changelog-rilasci-004 0.402
2. manuale-lumadesk-003 0.392

→ Per installare la scrivania: 1. cacciavite a croce… 3. le viti M8… [manuale-lumadesk-002]

retrieval su "installo": il montaggio della scrivania

rifiuto | fatto sbagliato | fonte sbagliata

Il generatore ricorda (la storia è nel prompt), il retrieval no, e la citazione finisce a caso sui chunk che ha davanti. Risposta giusta con fonte sbagliata è peggio del rifiuto.

esempi del docente, misurati sulla soluzione
i vostri cambiano nella forma, non nel tipo di errore



Lab 2: query condensation

SCRIVETE VOI — da soli, con affiancamento

`working/chat.py` · `TODO 3` · `60 minuti` · consegna in `CONSEGNA.md`

si rientra per il conto delle chiamate

`TODO 3 · condensa(storia, domanda)`

una chiamata al modello: la conversazione, la nuova domanda, e torna la domanda riscritta

`poi, dentro rispondi_in_chat()`

la riscritta va a rag.recupera() e a rag.genera(); in storia resta l'originale

`FATTO QUANDO`

Sul dialogo dove prima usciva il chunk sbagliato, compare la riga `↳ cercato: Qual è il prezzo della versione wide della LumaDesk?`, tra i chunk c'è `listino-catalogo-002` e la risposta lo cita. Poi gli altri dialoghi, uno alla volta: prima di ogni follow-up, chiedetevi se senza condensation fallirebbe. Chi finisce prima: follow-up propri sempre più vaghi, e si legge la riga cercato.



Dopo il Lab 2: cosa cambia e cosa costa

Dialogo	Nudo	Condensato	Chiamate
«E la versione wide?»	Non lo trovo nei documenti.	949 euro [listino-catalogo-002]	3 → 5
«Perché no?» (il braccio sul piano fisso)	Non lo trovo nei documenti.	spessore insufficiente [ticket-supporto-008]	3 → 5
«E se l'ho comprata con un codice promozionale?»	14 giorni, buono acquisto	14 giorni, buono acquisto	3 → 5 per niente

`COSTO`

un dialogo di tre turni: da $0,0003-0,0005 a $0,0004-0,0006. Quasi il doppio delle chiamate, sempre sotto il millesimo di dollaro.

`non sempre serve`

se il follow-up porta con sé le parole giuste, il retrieval nudo lo trova. Oggi si condensa sempre; decidere caso per caso è materia del Modulo 4.



Pausa

15 minuti

Al rientro Lab 3: riassunto e sessioni.

`working/chat.py · TODO 4-5`



Lab 3: summary memory e sessioni

SCRIVETE VOI — da soli, con affiancamento

`working/chat.py` · `TODO 4-5` · `50 minuti` · `consegna in CONSEGNA.md`

si rientra per il confronto buffer / riassunto

`TODO 4` · `riassumi(storia)`

sopra budget, tutto tranne l'ultimo turno diventa un riassunto; al posto di tronca() quando CONFIG["memoria"] è "riassunto" (exit, si cambia la riga, si rilancia)

`TODO 5` · `salva_sessione()` · `carica_sessione()`

un JSON per sessione in sessioni/; il main chiede il nome all'avvio e salva a ogni turno

`FATTO QUANDO`

Con "riassunto" la riga `[riassunto] …` compare quando la storia sfora e la chat continua. Poi exit, riavvio con lo stesso nome — `sessione 'prova': 4 messaggi ripresi` — e un follow-up sul turno di prima funziona ancora.



Buffer vs riassunto sullo stesso dialogo

Sette turni, budget 350: LD-200 · postazione di casa · contributo · garanzia · calibrazione · codici di errore · «Tornando alla scrivania di cui abbiamo parlato all'inizio: quanto pesa?»

```
memoria: buffer
turno 1 → 699 euro · … · turno 4 → 274 token
turno 5 → [buffer] tolti i 4 messaggi più vecchi
turno 7 · ↳ cercato: Quanto pesa la scrivania di cui abbiamo parlato all'inizio?
→ Non lo trovo nei documenti.
$0,0014 · 13 chiamate
```

```
memoria: riassunto
turno 1 → 699 euro · … · turno 4 → 268 token
turno 5 → [riassunto] La LumaDesk LD-200 costa 699 euro [listino-catalogo-002]. Per allestire una postazione… Il contributo va richiesto entro 60 giorni…
turno 7 · ↳ cercato: Quanto pesa la scrivania LumaDesk LD-200?
→ La LumaDesk LD-200 pesa 34 kg [manuale-lumadesk-011].
$0,0020 · 16 chiamate (tre riassunti)
```

Al quinto turno la storia sfora. Il buffer perde i turni 1-2 e la condensation non sa più di quale scrivania si parla; il riassunto la tiene in una riga. Ogni riassunto riassume anche il precedente.

numeri del docente, misurati sulla soluzione
nel blocco flessibile (A) lo rifate sul vostro chat.py



Blocco flessibile

A scelta del docente, 35 minuti.

A

Buffer vs riassunto

il dialogo lungo della slide precedente, sul vostro chat.py, prima con "buffer" e poi con "riassunto": si legge insieme cosa manca nel riassunto rispetto ai turni originali.

B

Q&A aperto

anche su Modulo 3 · Lezione 1-2: l'anatomia, il catalogo dei failure, CONFIG e i chunk usati.

C

Esercizio extra

stesso dialogo con buffer e riassunto: a quale turno il buffer dimentica il primo argomento.

```
CONFIG["budget_token"] = 150
```



La prossima lezione

Modulo 3 · Lezione 3
rag.py con genera() · chat.py

→

Modulo 3 · Lezioni 4–9 · single-turn, sul retrieval
hybrid search · re-ranking · debugging · tool calling · RAG + tools — rag.genera() in aikit/

→

Modulo 3 · Lezione 12
chat.py in aikit/

> Il `rag.py` di oggi, con `genera()`, è quello che trovate in `aikit/` dalla prossima lezione; `chat.py` resta a ognuno. La prossima lezione (Modulo 3 · Lezione 4, hybrid search) si torna a una domanda alla volta: da qui a Modulo 3 · Lezione 9 si lavora sul retrieval. La memoria torna nel toolkit a Modulo 3 · Lezione 12, quando il golden set avrà i follow-up.



Grazie

prossima lezione: Modulo 3 · Lezione 4 — Advanced Retrieval I: Hybrid Search

